Computer Organization and System Programming Project Milestone 1:

This project aims to design a basic computer system using Logisim, focusing on the integration of a memory unit, a set of registers, an arithmetic logic unit (ALU), and a common bus structure. The system is built around a shared 16-bit bus controlled by selection lines, enabling data transfer between all components in a structured and controlled manner. We will explain how it works in part B in the report:

Explaining the Architecture:

1) The bus

The circuit consists of 1 memory, 6 registers and an ALU, each having input and output lines connected to the bus (excluding the ALU). The output lines are all connected to an 8x1 Multiplexer (which is always enabled) with select lines S2 S1 S0, the selection of the bus is as follows:

S_2	S_1	S_0	Selection
0	0	1	Memory
0	1	0	AR
0	1	1	PC
1	0	0	DR
1	0	1	AC
1	1	0	IR
1	1	1	TR

When S2 S1 S0 = 0 0 0, it is unused so in the MUX it is left Unused.

The output of the 8x1 Multiplexer is connected to the memory and each of the registers excluding the AC, which will be discussed further on.

Data is taken from the registers, and depending on what is asked, the bus selects from the register/memory and uses it in the task required.

Each register has 16 bits, with the memory having an address width of 8 bits and data width of 16 bits.

2) Memory

The architecture consists of a 256 × 16-bit memory, so 8 bits for the address and 16 for the data, the memory has 7 inputs and 1 output, the inputs are:

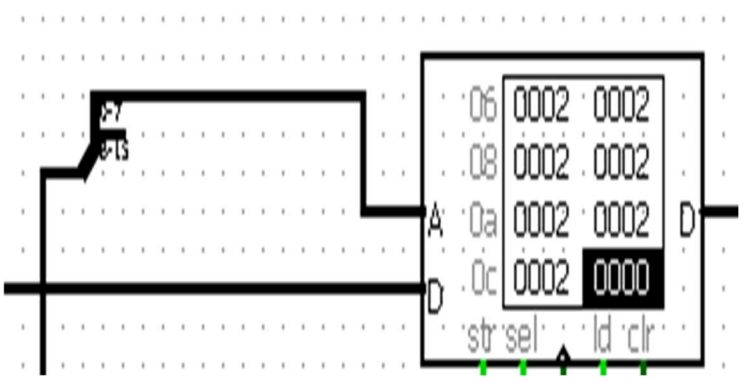
1. A, being the location (address) accessed in the memory. (8 bits)
2. D, being the input data to be stored in the memory. (16 bits)

Computer Organization and System Programming Project Milestone 1:

3. str, being the store, if it is 1 it stores input D in location A, if 0 it reads data at in location A. (1 bit)
4. sel, being the chip select, meaning if it is 1 the memory is on, if it is 0 it is off. (1 bit)
5. ld, being the load, if 1 it loads the data and outputs it, if 0 it doesn't. (1 bit)
6. clr, being the clear, when 1 it clears the data in the memory, when 0 it doesn't. (1 bit)
7. clock, it synchronizes the memory and the registers so the work at the same time when ticked.

We connected the inputs to a number of pins and constants:

1. The input A is taken from the address register, however there is a problem, the address register consists of 16 bits whereas A requires only 8 bits, to solve this problem we use a Splitter. A Splitter splits data per the required bits. To split the data from the AR, we used the Splitter and split it into to parts, 0-7 and 8-15, we only required the 0-7 bits, so we connect them to A and leave the rest.
   Here is the demonstration:



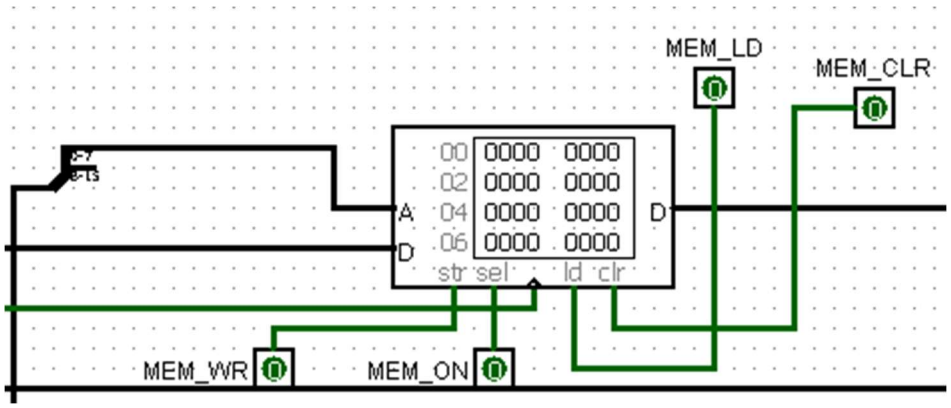
Selection: Splitter	
Facing	East
Fan Out	2
Bit Width In	16
Appearance	Left-handed
Bit 0	0 (Top)
Bit 1	0 (Top)
Bit 2	0 (Top)
Bit 3	0 (Top)
Bit 4	0 (Top)
Bit 5	0 (Top)
Bit 6	0 (Top)
Bit 7	0 (Top)
Bit 8	1 (Bottom)
Bit 9	1 (Bottom)
Bit 10	1 (Bottom)
Bit 11	1 (Bottom)
Bit 12	1 (Bottom)
Bit 13	1 (Bottom)
Bit 14	1 (Bottom)
Bit 15	1 (Bottom)

2. The input D is connected to the bus (output from the 8x1 MUX). (16 bits)
3. The input str is connected to a pin named MEM_WR. (1 bit)
4. The input sel is connected to a pin named MEM_ON. (1 bit)
5. The input ld is connected to a pin named MEM_LD. (1 bit)
6. The input clr is connected to a pin named MEM_CLR. (1 bit)
7. The clock input is connected to the clock CLK.

Computer Organization and System Programming Project Milestone 1:

The output of the memory is connected to the bus (8x1 MUX).

This is the final image of the memory:



3) Registers:

The circuit consists of 6 registers: AR, PC, DR, AC IR, TR. Each register is 16 bits wide. In this project registers are implemented as counters.

Each register has 5 inputs and 2 outputs, the inputs are:

1. D, being the data input into the register. (16 bits)
2. ld, being the load, when it is 1 it loads input D into the register, when it is 0 it doesn't load. (1 bit)
3. ct, being the count, when it is 1 and the ld is 0 it increments the value inside of the register, when it is 1 and the ld is 1 it decrements the value inside of the register, when it is 0 it doesn't increment nor decrement. (1 bit)
4. clr, being the clear, when it is 1 it clears the data in the register, when it is 0 it doesn't clear. (1 bit)
5. clk, being the clock, it synchronizes the memory and the registers so the work at the same time when ticked.

The two outputs are:

1. Q being the data output from the register. (16 bits)
2. Carry, when there is overflow from incrementing the data inside the register, it is , when there isn't it is 0. (1 bit)

Computer Organization and System Programming Project

Milestone 1:

Each register takes inputs as follows:

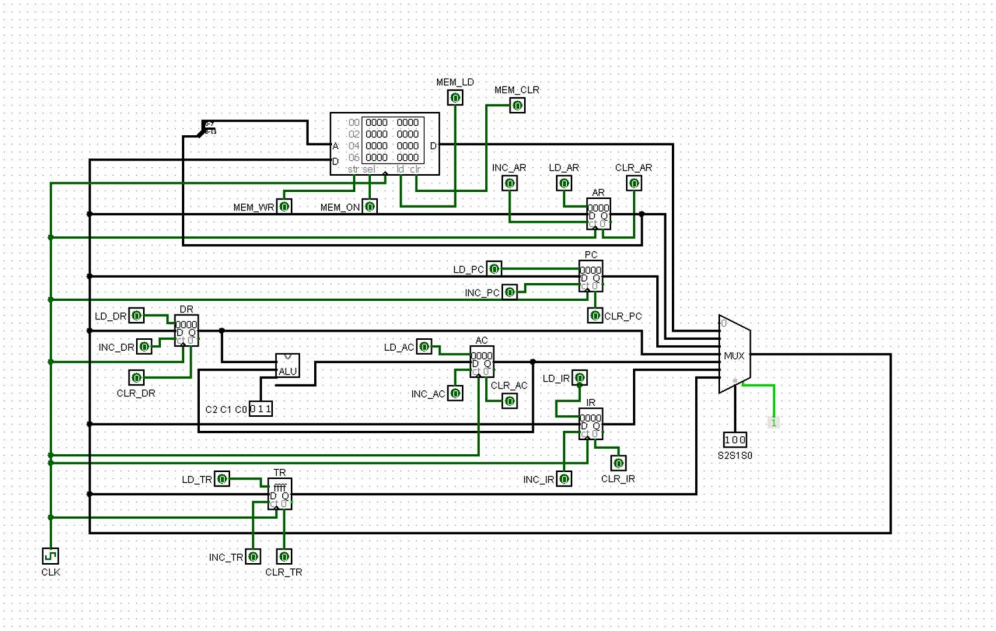
1. D from the bus (Excluding the AC). (16 bits)
2. ld from a unique pin for each register labelled as LD_ followed by register name, for example (LD_AR, LD_PC, LD_DR, LD_AC, LD_IR, LD_TR). (1 bit)
3. ct from a unique pin for each register labelled as INC_ followed by register name, for example (INC_AR, INC_PC, INC_DR, INC_AC, INC_IR, INC_TR). (1 bit)
4. clr from a unique pin for each register labelled as CLR_ followed by register name, for example (CLR_AR, CLR_PC, CLR_DR, CLR_AC, CLR_IR, CLR_TR). (16 bits)
5. The clock input is connected to the same clock CLK for each register.

For the AC, the input D is taken from the ALU.

The outputs are as following:

1. The output Q is connected to the bus for all registers. (16 bits)
2. The carry is unused.

Here is the circuit for the registers including the memory and ALU:



Computer Organization and System Programming Project Milestone 1:

4) ALU:

In this design, the ALU takes two inputs, A and B (16 bits), which are provided by the Data Register (DR) and the Accumulator (AC), respectively. The operation performed by the ALU is controlled by a 3-bit control signal (C2 C1 C0), allowing it to execute multiple functions.

The implemented ALU supports operations such as data transfer, addition, subtraction, multiplication, division, complement, and XOR. The result of the selected operation is forwarded to the output, which can then be loaded into the AC depending on the requirement of the task.

C2	C1	C0	Function
0	0	1	Transfer A (coming from DR)
0	1	0	Add A + B
0	1	1	Subtract A − B
1	0	0	Multiply A x B
1	0	1	Divide A / B
1	1	0	Complement A
1	1	1	A XOR B

When C2 C1 C0 are 0 0 0 respectively, it is unused.

When adding, we use an Adder which takes inputs A, B and C in (C in is 1 bit and set to 0), and outputs the result to the 8x1 MUX, and the C out (carry out is unused).

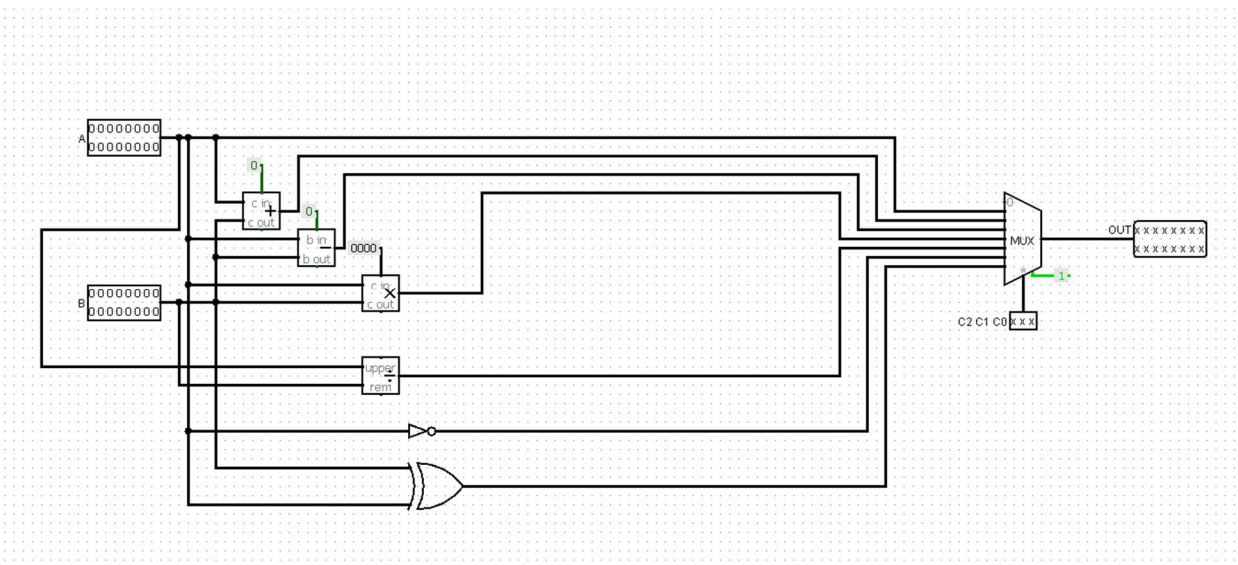
Computer Organization and System Programming Project Milestone 1:

When subtracting, we use a Subtractor which takes inputs A, B and b in (b in is 1 bit and set to 0), and outputs the result to the 8x1 MUX, and the b out (borrow out is unused).

When multiplying, we use a Multiplier which takes inputs A, b and C in (C in is 16 bits and set to 0000), and outputs the result to the 8x1 MUX, and the C out (carry out is unused).

When dividing we use a Divisor which takes inputs Dividend Upper, A and B (Dividend upper is 16 bits and set to 0000), and outputs the result to the 8x1 MUX, and the rem out (rem is unused).

Here is the ALU circuit:



Computer Organization and System Programming Project Milestone 1:

Part A:-

1) M[2] = M[0] / –M[1]:

Cycle 1: DR ← M[AR]

This copies the data in memory at the address stored in AR, which is address 0 and loads it into the DR.

DR = M[0] = 10 (000a in hexadecimal).

Cycle 2: TR ← DR

Copies the content of the DR and loads it into the TR for later use.

TR = 10 (000a in hexadecimal, and saving M[0] for later).

Cycle 3: AR ← AR + 1

Incrementing AR to access next address in the memory.

AR = 1 (0001 in hexadecimal).

Cycle 4: DR ← M[AR]

This copies the data in memory at the address stored in AR, which is address 1 and loads it into the DR.

DR = M[1] = –5 (fffb in hexadecimal).

Cycle 5: AC ← DR '

Using the ALU, we select C2 C1 C0 = 110 to complement input A, which is the value in the DR register, and load it into the AC.

AC = 4 (0004 in hexadecimal).

Cycle 6: AC ← AC + 1

Incrementing AC, thereby converting the data in the M[1] to its negative value.

AC = 0005 (in hexadecimal) = 5 = –M[1].

Cycle 7: DR ← TR

Computer Organization and System Programming Project Milestone 1:

Now we load the data from the TR into the DR.

DR = 10 (000a in hexadecimal).

Cycle 8: AC ← DR / AC

Using the ALU we select C2 C1 C0 = 101 to divide input A by B (DR by AC), thereby doing the step M[0] / -M[1] and loading the output into the AC.

AC = 10 / 5 = 2 (0002 in hexadecimal).

Cycle 9: AR ← AR + 1

Incrementing AR to access address 2 in the memory

AR = 2 (0002 in hexadecimal).

Cycle 10: M[AR] ← AC

Now we write the contents of the AC into the memory with the address from the AR.

M[2] = 2 (0002 in hexadecimal).

Thus we completed steps for 1.

2) M[4] = M[2] – (M[3] + 1):

Cycle 11: DR ← M[AR]

This copies the data in memory at the address stored in AR, which is address 2, and loads it onto the DR.

DR = M[2] = 2 (0002 in hexadecimal).

Cycle 12: TR ← DR

Copies the content of the DR and loads it to the TR for later use.

TR = 2 (0002 in hexadecimal, and saving M[2] for later).

Cycle 13: AR ← AR + 1

Incrementing the AR to access next address in the memory.

Computer Organization and System Programming Project Milestone 1:

AR = 3 (0003 in hexadecimal).

Cycle 14: DR ← M[AR]

This copies the data in memory at the address stored in AR, which is address 3 and loads it onto the DR.

DR = M[3] = 2 (0002 in hexadecimal).

Cycle 15: AC ← DR

Using the ALU, we select C2 C1 C0 = 001 to transfer input A, which is the value in the DR register, and load it onto the AC.

AC = 2 (0002 in hexadecimal).

Cycle 16: AC ← AC + 1

Incrementing AC, thereby adding 1 to the value of M[3].

AC = 0003 (in hexadecimal) = 3 = M[3] + 1.

Cycle 17: DR ← TR

Now we load the data from the TR onto the DR.

DR = 2 (0002 in hexadecimal).

Cycle 18: AC ← DR - AC

Using the ALU we select C2 C1 C0 = 011 to subtract input B from A (DR - AC), thereby doing the step M[2] - (M[3] + 1) and load the output into AC.

AC = 2 - (2+ 1) = -1 (ffff in hexadecimal).

Cycle 19: AR ← AR + 1

Incrementing the AR to access address 4 in the memory

AR = 4 (0004 in hexadecimal).

Cycle 20: M[AR] ← AC

Now we write the contents of the AC onto the memory with the address from the AR.

M[4] = -1 (ffff in hexadecimal). Thus we completed steps for 2.

Computer Organization and System Programming Project

Milestone 1:

Testing:-

For testing, first input onto addresses in the memory as follows:

Address	Value
0	10
1	-5
2	0
3	2
4	0

Address 00: should be decimal value 10, so convert into hexadecimal it should be 000A

Address 01: should be decimal value -5, so convert into hexadecimal it should be FFFB

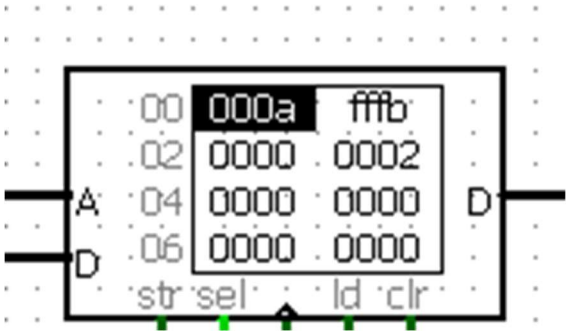
Address 02: should be decimal value 0, so convert into hexadecimal it should be 0000

Address 03: should be decimal value 2, so convert into hexadecimal it should be 0002

Address 04: should be decimal value 0, so convert into hexadecimal it should be 0000

AR register should be initialized to 0.

It should look like this;



Computer Organization and System Programming Project Milestone 1:

Part B:-

To solve part A using the cycles here are the steps:-

After every step make sure you tick on the clock (1 cycle).

1) M[2] = M[0] / –M[1]:

Cycle 1: DR ← M[AR]

Set LD_DR, MEM_ON, MEM_LD to 1.

Set S2 S1 S0 to 0 0 1 respectively.

Set everything else to 0.

Cycle 2: TR ← DR

Set LD_TR to 1.

Set S2 S1 S0 to 1 0 0 respectively.

Set everything else to 0.

Cycle 3: AR ← AR + 1

Set INC_AR to 1.

Set everything else to 0.

Cycle 4: DR ← M[AR]

Set LD_DR, MEM_ON, MEM_LD to 1.

Set S2 S1 S0 to 0 0 1 respectively.

Set everything else to 0.

Cycle 5: AC ← DR '

Set C2 C1 C0 to 1 1 0 respectively.

Set LD_AC to 1.

Set everything else to 0.

Computer Organization and System Programming Project Milestone 1:

Cycle 6: AC ← AC + 1

Set INC_AC to 1.

Set everything else to 0.

Cycle 7: DR ← TR

Set LD_DR to 1.

Set S2 S1 S0 to 1 1 1 respectively.

Set everything else to 0.

Cycle 8: AC ← DR / AC

Set C2 C1 C0 to 1 0 1 respectively.

Set LD_AC to 1.

Set everything else to 0.

Cycle 9: AR ← AR + 1

Set INC_AR to 1.

Set everything else to 0.

Cycle 10: M[AR] ← AC

Set MEM_WR, MEM_ON to 1.

Set S2 S1 S0 to 101 respectively.

Set everything else to 0.

2) M[4] = M[2] – (M[3] + 1):

Cycle 11: DR ← M[AR]

Set LD_DR, MEM_ON, MEM_LD to 1.

Set S2 S1 S0 to 0 0 1 respectively.

Set everything else to 0.

Cycle 12: TR ← DR

Computer Organization and System Programming Project Milestone 1:

Set LD_TR to 1.

Set S2 S1 S0 to 1 0 0 respectively.

Set everything else to 0.

Cycle 13: AR ← AR + 1

Set INC_AR to 1.

Set everything else to 0.

Cycle 14: DR ← M[AR]

Set LD_DR, MEM_ON, MEM_LD to 1.

Set S2 S1 S0 to 0 0 1 respectively.

Set everything else to 0.

Cycle 15: AC ← DR

Set LD_AC to 1.

Set C2 C1 C0 = 0 0 1 respectively.

Set everything else to 0.

Cycle 16: AC ← AC + 1

Set INC_AR to 1.

Set everything else to 0.

Cycle 17: DR ← TR

Set LD_DR to 1.

Set S2 S1 S0 to 1 1 1 respectively.

Set everything else to 0.

Cycle 18: AC ← DR - AC

Set LD_AC to 1.

Set C2 C1 C0 = 011

Computer Organization and System Programming Project Milestone 1:

Set everything else to 0.

Cycle 19: AR ← AR + 1

Set INC_AR to 1.

Set everything else to 0.

Cycle 20: M[AR] ← AC

Set MEM_WR, MEM_ON to 1.

Set S2 S1 S0 to 1 0 1 respectively.

Set everything else to 0.